

How Praxis Designs Physical AI Systems: Method and Evidence from Seven Reference Architectures

What it takes to design a system for the physical world, and how Praxis does it: the inputs, the eight reasoning lenses, the rules that hold on every design, and the evidence from seven published reference architectures.

CodeNinja Engineering Team · Umar Bilal

CodeNinja · 2026-10-04 · CC BY 4.0 · Web edition: <https://muhammadumar89.github.io/codeninja-research/praxis-method/>

Abstract. Designing physical AI, for a port, a grid, a mill or a construction site, has always needed deep domain expertise: what to sense, where each model may run, which rules bind, what the hardware must hold and who must approve each action. Praxis is CodeNinja's platform for designing such systems. It reads an operator's requirement in the operator's own words, loads the sector's knowledge as context, reasons through eight lenses (first principles, case studies, rules and regulations, approach, tooling and recency, history, domain fusion, and hardware and equipment) and renders one validated plan into a complete design: scope and rollout, layered architecture, object model, model and equipment register, cost and a live simulation. Across the seven designs published in the Vertical-Driven Architectures series, covering 4 sectors in 3 countries, Praxis listed 7,672 records as candidate context and read 872 of them in full, its lenses cited 142 sources, and 8 times a lens found nothing citable and said so instead of filling the gap. 5 of the seven designs cover every recorded requirement through a rollout gate; the other two print their lower coverage as counted. Every design ships its object model as a `hyper-ontology/1` package that [Hyper Ontology](#) imports to stand up a living system.

1. The problem: physical systems need a domain engineer's judgement

A software system can be designed from its data model outward. A physical AI system cannot. Its first decisions are physical: whether detection must happen at the edge because the link drops in the storm that matters, whether a thermal camera crosses an export threshold, whether a 753 billion parameter model fits the GPUs the country can receive, whether a model may ever write to life-safety equipment. Those decisions have traditionally been made by engineers with a decade in one sector, and they are the reason designs for ports, grids, mills and sites take months to write and are rarely shared.

Praxis exists to make that judgement explicit, reproducible and fast: to design the system a ten-year domain engineer would, show the reasoning, and cite what justified each choice.

2. What Praxis takes in

The requirement, in the operator's own words. Praxis reads the ask as written, often a published request for proposals, and does not paraphrase it before reasoning. The forward deployed engineer may pin what the requirement leaves open: the sector, a reference architecture, and the country the design is written for.

The room. Praxis assigns the family and industry, then lists the sector's knowledge as candidate context: regulations, tooling and model records, approaches, case studies, history and hardware entries. Across the seven designs the room held between 821 and 1,329 records, and Praxis read between 115 and 133 of them in full. Nothing is ranked or filtered by keyword: the model reads and decides what applies. Nothing is fine-tuned: the knowledge rides as context, so a design in a new sector needs a thicker room, not a new model.

3. How Praxis reasons: eight lenses

LENS	THE QUESTION IT ANSWERS
First principles	Why must the design take this shape, from the physics and the operation itself?
Case studies	How did comparable operations handle this problem, and what failed?
Rules and regulations	Which rules of this country and sector bind the design, including export controls?
Approach	Which patterns fit, which are set aside, and why?
Tooling and recency	Which models, runtimes and products are current and correctly licensed today?
History	What did earlier designs in this sector learn?
Domain fusion	Where do two disciplines meet in one decision?
Hardware and equipment	Which compute, sensing and field equipment does the design land on, and how is it sized?

Table 1. The eight lenses. Each returns either a contribution with sources the design can cite, or a gap stated in a sentence.

Three rules hold on every design. **AI reasons, tools generate:** the model writes one validated plan, and code renders every document, figure and simulation from it, so one plan always yields the same output. **No claim without a record:** a model, regulation or pattern survives into a design only if a source supports it, and a lens with nothing to cite says so. **A person on every write:** designs recommend; a named person approves anything that changes a plan, a schedule or a piece of equipment.

4. What Praxis produces

From one plan Praxis renders a scope baseline whose phases carry item counts and gates rather than durations; a layered architecture from sources through adapters, one object model, inference tiers and surfaces; the object model as a hyper-ontology/1 package; a model and equipment register in which every GPU class follows from memory arithmetic (parameters times bytes per parameter, plus a factor for cache and activations, against the memory of the class the country can receive); a three-year cost comparison from cited public prices; a live simulation; a proposal and functional specification; and the research paper. Chapter 11 of every paper in the series is the reasoning record of that design.

5. Evidence from seven designs

DESIGN	SECTOR	COUNTRY	LISTED	READ IN FULL	CITED	GAPS	PHASES / ITEMS	REQUIREMENTS COVERED
Sovereign HSE Watch	oil and gas	Pakistan	1,243	116	28	1	3 / 17	0 of 12
Feeder Firewatch	energy and utilities	United States	1,329	115	20	0	2 / 15	5 of 5
Terminal Pulse	maritime and ports	United States	821	131	26	0	3 / 14	5 of 5
Structure Phase Watch	heavy industry and construction	Saudi Arabia	1,142	127	25	1	3 / 18	2 of 7
Steel Count Ledger	heavy industry and construction	Pakistan	1,043	133	17	3	3 / 15	15 of 15
Factory Fire Watch	heavy industry and construction	Saudi Arabia	1,272	119	16	1	4 / 19	5 of 5
Port Twin	maritime and ports	United States	822	131	10	2	4 / 18	16 of 16

Table 2. What Praxis read and cited for each design, and the coverage each design prints. Listed and read in full are the room; cited is the sum across the eight lenses; gaps counts lenses that found nothing citable.

	Sovereign HSE Watch	Feeder Firewatch	Terminal Pulse	Structure Phase Watch	Steel Count Ledger	Factory Fire Watch	Port Twin
First principles	4	1	3	4	1	2	1
Case studies	gap	3	3	3	gap	gap	1
Rules and regulations	4	1	2	2	gap	1	gap
Approach	5	3	3	4	3	3	2
Tooling and recency	5	3	5	4	5	3	3
History	2	1	2	gap	gap	1	1
Domain fusion	2	2	2	2	2	2	2
Hardware and equipment	6	6	6	6	6	4	gap

Figure 1. Sources each lens cited, per design. A dashed cell is a lens that found nothing citable in the room and said so.

What the evidence shows

Praxis reads about one record in ten in full. Across the series it read 872 of 7,672 listed records (11 percent). The rest stay available as titles; the model chooses which to open, and the choice is recorded.

Hardware is never an afterthought. The hardware and equipment lens cited four or more sources in 6 of the seven designs. It is the lens that fixes whether a frontier model runs on one node of eight 141 GB GPUs, whether vision runs on a Jetson Orin class edge box in a solar enclosure, or whether the design buys no compute at all, as in the port and factory designs.

Gaps are stated, not filled. 8 lens results across the series are gaps: case studies 3, rules and regulations 2, history 2, hardware and equipment 1. They mark where the sector's knowledge is thin today, most often in case studies, and they are printed in the paper rather than papered over with a plausible source.

Coverage is counted, not asserted. 5 designs close every recorded requirement through a rollout gate. Structure Phase Watch covers two of seven and states that five sit outside its baseline as later scope. Sovereign HSE Watch prints none of twelve as covered by a gate. Both numbers are the platform's count, published as counted.

6. Where people stay in the loop

The forward deployed engineer pins what the requirement leaves open and reviews every design before it leaves. Every design names its write paths and the person who approves each. Before publication a separate gate reads every paper, figure and file for operator names, near identifiers and claims that will not survive scrutiny, such as an export control statement for the wrong country group, and the paper does not ship until it passes.

7. Limits

The designs are reference architectures, not quotations: hardware prices are public list prices on a stated date, and counts such as users or installation points are assumptions printed where they are used. The quality of a design follows the thickness of the sector's room; the gaps in Figure 1 are where it is thinnest today. Praxis is in beta, used in house by CodeNinja's forward deployed engineers; access for outside teams is by request at <https://muhammadumar89.github.io/codeninja-research/praxis/>.

References

1. CodeNinja Engineering Team and Umar Bilal. 2026. *Sovereign HSE Watch: Predictive Risk and Early Warning on an HSE Control and Command Platform*. CodeNinja. <https://doi.org/10.5281/zenodo.23119714>
2. CodeNinja Engineering Team and Umar Bilal. 2026. *Feeder Firewatch: Live Ignition and Outage Risk for Every Distribution Feeder*. CodeNinja. <https://doi.org/10.5281/zenodo.23119325>
3. CodeNinja Engineering Team and Umar Bilal. 2026. *Terminal Pulse: Predicted Truck Turn Time and Live Yard Sight for a Container Terminal*. CodeNinja. <https://doi.org/10.5281/zenodo.23119348>
4. CodeNinja Engineering Team and Umar Bilal. 2026. *Structure Phase Watch: Live Production, Crane and Delivery Evidence for Every Pour on a Construction Site*. CodeNinja. <https://doi.org/10.5281/zenodo.23126448>
5. CodeNinja Engineering Team and Umar Bilal. 2026. *Steel Count Ledger: Independently Counted Production for Every Steel Mill in Pakistan*. CodeNinja. <https://doi.org/10.5281/zenodo.23126563>
6. CodeNinja Engineering Team and Umar Bilal. 2026. *Factory Fire Watch: Read-Only Smart Fire Protection Monitoring for Every High-Risk Factory*. CodeNinja. <https://doi.org/10.5281/zenodo.23126565>
7. CodeNinja Engineering Team and Umar Bilal. 2026. *Port Twin: One Governed Digital Twin for Every Asset, Feed and Dollar*. CodeNinja. <https://doi.org/10.5281/zenodo.23126431>